

BitFlip

CloudSherpa

Architecture Decision Records

Contents

1	Introduction	2
2	Records	3
2.1	ADR0001	3
2.2	ADR0002	4
2.3	ADR0003	5
2.4	ADR0004	5
2.5	ADR0005	6
2.6	ADR0006	7

1 | Introduction

This document serves as a log of all major architectural decisions made by team Bit-flip for the CloudSherpa project. All architecture decision records added shall be immutable after addition, and may only be amended using the status field to refer to a subsequent ADR. Records within this document shall follow the format below.

ADR <number>

Title: [Brief, descriptive title of the decision]

Status: [Proposed / Accepted / Superceded by ADR <number>]

Context: [*What impacted this decision*]

Alternatives: [*Options considered for the solution*]

1. Option 1 and brief advantages and disadvantages
2. Option 2 and brief advantages and disadvantages

Decision: [*Descriptive enough that system compliance can be checked*]

Consequence: [*Tradeoffs of the decision*]

- + Positive consequence one
- + Positive consequence two
- Negative consequence one

2 | Records

2.1 ADR0001



ADR 0001

Title: Ingestion Scheduler job management

Status: Accepted

Context: Ingestion jobs from multiple cloud accounts must be ingested at varying times dependent on user-selected intervals. These events must not interfere with the rest of the subsystem's functioning. Failure conditions must be handled to ensure resiliency and avoid data loss in the case of external outages.

Alternatives:

1. Use Postgress listen/notify to create one thread per due account (could lead to thread explosion and deployment costs, no outage protection, satisfies independent functioning, tightly coupled to database).
2. Use database polling with timestamped fields and a thread pool to queue due account jobs. (No thread explosion, outage protection, no retry logic, manual retry logic and exception handling required).
3. Use database polling with timestamped fields and a background job manager (JobRunr) to manage threads. (No thread explosion, outage protection, automatic retry attempts, detailed logs maintained, additional storage required, dependency)

Decision: Database polling with timestamped fields and JobRunr background job manager

Consequence:

- + Threads are managed by JobRunr to avoid thread explosion.
- + JobRunr manages retry logic automatically for resiliency.
- + Detailed logging is available through Jobrunr generated tables.
- + Polling allows jobs to be handled multiple times in case of failure
 - JobRunr handles retries uniformly, potentially limiting future custom handling
 - JobRunr adds overhead and logs may incur additional storage costs

2.2 ADR0002



ADR 0002

Title: PostgreSQL LISTEN/NOTIFY for inter-service communication

Status: Accepted

Context: The ingestion subsystem must notify the service subsystem when metrics have been added. The communication mechanism should minimise deployment complexity while remaining sufficiently reliable for the expected workload.

Alternatives:

1. Use an event broker (e.g. Kafka or RabbitMQ). (Mature event-driven communication, durable messaging, scalable, increased operational complexity and additional infrastructure).
2. Use PostgreSQL LISTEN/NOTIFY. (Simple deployment, no additional infrastructure, lightweight, notifications are transient and less suitable for large-scale event streaming).

Decision: Use PostgreSQL LISTEN/NOTIFY for communication between the ingestion and service subsystems.

Consequence:

- + Reduces operational complexity by avoiding additional messaging infrastructure.
- + Reduces deployment size and simplifies maintenance.
- + Integrates naturally with the existing PostgreSQL database.
- Notifications are not durable and may be missed if consumers are unavailable.
- Less suitable than dedicated message brokers for future high-volume event streaming.

2.3 ADR0003

ADR 0003

Title: Server-Sent Events for real-time dashboard updates

Status: Accepted

Context: Users should metric updates without manually refreshing the dashboard or generating unnecessary requests.

Alternatives:

1. Poll the server periodically. (Simple implementation, wasted requests when no updates exist, increased server load).
2. Use Server-Sent Events (SSE). (Real-time updates over a persistent HTTP connection, lower network overhead, persistent connections must be maintained).

Decision: Use Server-Sent Events (SSE) to push ingestion status updates to the dashboard.

Consequence:

- + Eliminates unnecessary polling requests.
- + Provides near real-time updates to users.
- Persistent HTTP connections consume server resources.
- Connections must be re-established after network interruptions.

2.4 ADR0004

ADR 0004

Title: Cloud billing exports for cost analysis

Status: Accepted

Context: Cloud resource costs can either be approximated from usage metrics or obtained directly from provider billing exports. Accurate historical costs are required for reporting.

Alternatives:

1. Approximate costs using collected usage metrics. (Near real-time estimates, numerous unknown pricing variables reduce accuracy, metric specific logic implementation).
2. Use cloud provider billing exports. (Accurate provider-calculated costs, delayed availability due to billing export schedules).

Decision: Use provider billing exports as the source of cloud cost information.

Consequence:

- + Provides accurate provider-calculated billing values.
- + Avoids implementing and maintaining complex pricing logic.
- + Produces consistent historical reporting.
- Users accept latency before costs become visible.

2.5 ADR0005



ADR 0005

Title: Use of foundational prediction models

Status: Accepted

Context: CloudSherpa provides projections of future metric values of cloud resource utilisation based on historical data. The prediction model could either be developed and trained specifically for the system or utilise an existing foundational model.

Alternatives:

1. Develop a custom prediction model. (Highly sophisticated models could achieve higher quality predictions if provided sufficient data, significant development effort, extensive training data required, skill requirement with parameter tuning or method selection).
2. Use an existing foundational language model. (High-quality predictions based on large datasets, rapid development, dependency on third-party model providers).

Decision: Use a foundational prediction model to predict future metric values based on historical data.

Consequence:

- + Significantly reduces development effort.
- + Produces high-quality predictions based on extensive training set.
- Creates dependency on external model providers.
- Less tuning and optimization according to specific customer profiles possible.

2.6 ADR0006

ADR 0006

Title: Adopt a schema-per-tenant multi-tenancy architecture**Status:** Accepted

Context: CloudSherpa must support a growing number of tenants while maintaining logical data isolation, efficient query performance, and manageable operational overhead. The chosen multi-tenancy strategy should balance scalability, maintainability, and performance as the number of tenants and stored historical cloud metrics increases.

Alternatives:

1. Database-per-tenant. (Strong physical isolation and independent administration, however managing potentially thousands of databases significantly increases operational complexity, resource consumption, backups, monitoring, and schema migrations.)
2. Shared schema with tenant identifiers. (Simplified database administration with a single schema, however all tenant data resides in the same tables, resulting in larger indexes, tenant filtering on every query, increased risk of accidental cross-tenant data access, and reduced query locality.)
3. Schema-per-tenant. (Provides logical isolation while sharing the same database instance, resulting in smaller tenant-specific tables, improved query performance, and significantly lower administrative overhead than maintaining separate databases.)

Decision: Store each tenant's resource, metric and billing data within a dedicated PostgreSQL schema while sharing a common database instance and maintaining shared public tables for common data.

Consequence:

- + Provides strong logical isolation between tenants.
- + Avoids the operational overhead of managing potentially thousands of databases.
- + Queries execute against smaller tenant-specific tables, improving index efficiency and query performance.
- + Simplifies backups, deployments, and infrastructure management compared to database-per-tenant architectures.
- + Reduces the likelihood of accidental cross-tenant queries.
- Database schema migrations must be applied to every tenant schema.
- All tenants share the same underlying database resources, requiring future horizontal scaling if database capacity is exceeded.